MEDSTATS

MANUAL DEL DESARROLLADOR

DIAGNÓSTICO POR IMÁGENES

+

SISTEMA DE ESTERILIZACIÓN Y TRAZABILIDAD DE MATERIAL QUIRÚRGICO

OCTUBRE 2026

EQUIPO

Cavallo Mateo
Martinez Joaquin
Paulinsky Martina
Sotelo Tomas
Ponce Agustin
Baez Julian

INSTITUTO SUPERIOR DE FORMACIÓN TÉCNICA N°38

Hospital San Felipe
San Nicolás de los Arroyos

Índice

1. Introducción 3
2. Pila tecnológica 4
3. Arquitectura de la aplicación 5
   - 3.1 PatrónMVC 5
   - 3.2 Estructura de directorios 5
   - 3.3 Convenciones delproyecto 5
4. Seguridad y control de acceso 7
   - 4.1 Modelodeperfiles 7
   - 4.2 Middleware CheckRole 7
   - 4.3 Autorización dentro de controladores y vistas 7
   - 4.4 Otrasmedidas 8
5. Base de datos 9
   - 5.1 Diagrama entidad-relación · Trazabilidad 9
   - 5.2 Diccionario de datos · Trazabilidad 9
   - 5.3 Diagrama entidad-relación · Diagnóstico por Imágenes 11
   - 5.4 Diccionario de datos · DiagnósticoporImágenes 11
6. Módulo de Trazabilidad: implementación 13
   - 6.1 Componentes 13
   - 6.2 Métodos del controlador 13
   - 6.3 Máquina de estados 14
   - 6.4 Auditoría: retroceso sin pérdida de información 14
   - 6.5 Vistas y comportamiento del cliente 15
   - 6.6 Cómo extenderlo 15
7. Módulo de Diagnóstico por Imágenes: implementación 16
   - 7.1 Componentes 16
   - 7.2 Rutas y endpoints auxiliares 16
   - 7.3 Validación 17
   - 7.4 Descuento transaccional de insumos 17
   - 7.5 Estadísticas 18
   - 7.6 Exportación 18
   - 7.7 Cómo extenderlo 19
8. Mapa de rutas 20
   - 8.1 Trazabilidad (middleware auth + roles:admin,trazabilidad) 8.2 20
   - Diagnóstico por Imágenes (auth + roles:estudios_medicos) 8.3 20
   - Administración y soporte (roles:admin salvo indicación) 21
9. Capa de presentación 22
10. Instalación y despliegue local 23
    - 10.1 Requisitos 23
    - 10.2 Pasos 23
    - 10.3 Datos mínimos para operar los nuevos módulos 24
    - 10.4 Publicar el manual de usuario 24
    - 10.5 Despliegue en producción (lineamientos) 24
11. Estándares de codificación y flujo de trabajo 25
12. Pruebas 26

1. Introducción

Bienvenido al Manual del Desarrollador de **MedStats**, un sistema de gestión hospitalaria del Hospital San Felipe que en su última versión incorpora dos módulos: el **Sistema de Esterilización y Trazabilidad de Material Quirúrgico** y el módulo de **Diagnóstico por Imágenes**. Este documento proporciona a ingenieros de software, arquitectos y desarrolladores una guía clara sobre la estructura, la arquitec- tura y la lógica de la plataforma, garantizando una curva de aprendizaje rápida y una contribución efectiva al código.

Módulo de Trazabilidad

Resuelve un desafío operativo crítico: el control absoluto del instrumental quirúrgico. Cada caja se rastrea desde su armado y esterilización hasta su uso en el quirófano, con un registro auditable de su ciclo de vida (modelos `CajaQuirurgica` e `HistorialCaja`).

Módulo de Diagnóstico por Imágenes

Registra estudios de Rayos X y Tomografía (`EstudioMedico`), vinculados a paciente, médico y catálogo de estudios, y descuenta de forma transaccional los insumos de contraste del stock del servicio.

Alcance de este manual

- **Arquitectura y pila tecnológica** (capítulos 2 y 3).
- **Seguridad y control de acceso por perfiles** (capítulo 4).
- **Modelo de datos** con diagramas entidad-relación (capítulo 5).
- **Lógica de cada módulo**: controladores, modelos, reglas de negocio y vistas (capítulos 6 y 7).
- **Mapa de rutas** (capítulo 8) y **capa de presentación** (capítulo 9).
- **Guía de instalación y despliegue local** (capítulo 10), **estándares de código** (11) y **pruebas** (12).

Ya sea que su objetivo sea optimizar el registro de historiales, expandir los atributos de las cajas quirúrgicas, incorporar nuevas modalidades de imagen o mejorar la interfaz de seguimiento, este manual funciona como el mapa arquitectónico de MedStats. El repositorio es `github.com/AgustinPonceDev/MedStats` y la versión documentada corresponde a la rama `trazabilidad`.

2. Pila tecnológica

Capa	Tecnología	Detalle
Lenguaje / framework	**PHP ≥ 8.1** · **Laravel 10.50**	Núcleo MVC; Eloquent ORM, Blade, validación, middleware, migraciones.
Autenticación	Laravel Breeze 1.29 · Sanctum 3.3	Login, registro, recuperación de contraseña y verificación de email (`routes/auth.php`).
Base de datos	**MySQL / MariaDB** (utf8mb4)	Migraciones versionadas. Hay consultas con funciones propias de MySQL (`DATE_FORMAT, YEAR, WEEKDAY, REGEXP`).
Presentación	Blade + **Bootstrap 4.5** + **Tailwind** (build local)	Layouts en `resources/views/layouts`, parciales de sidebar y footer.
Interactividad	jQuery 3.6 · Select2 · DataTables · SweetAlert2 · Lucide	Selects encadenados, tablas ordenables/paginables, confirmaciones y íconos.
Gráficos y reportes	Chart.js · jsPDF + AutoTable · SheetJS (xlsx)	Estadísticas y exportación a PDF / Excel / impresión.
Empaquetado	**Vite 6** + laravel-vite-plugin + PostCSS	`resources/js/app.js` inicializa Axios y Alpine.js (scaffolding de Breeze).
Entorno local	Laragon (Windows) · Composer · Node 22	Servidor Apache/Nginx + MySQL; también funciona con `php artisan serve`.
Calidad	Laravel Pint 1.x · PHPUnit 10	Formateo PSR-12 y pruebas automatizadas.

> **NOTA** Los scripts y estilos de terceros (jQuery, DataTables, Select2, Chart.js, etc.) se sirven **localmente** desde `public/vendor`, de modo que el sistema funciona sin acceso a Internet dentro de la red del hospital.

3. Arquitectura de la aplicación

3.1 Patrón MVC

MedStats aplica el patrón **Modelo-Vista-Controlador** de Laravel, separando el acceso a datos (`app/Models`), la lógica de aplicación (`app/Http/Controllers`) y la presentación (`resources/views`).

La autenticación y la autorización se resuelven antes de llegar al controlador mediante middleware.

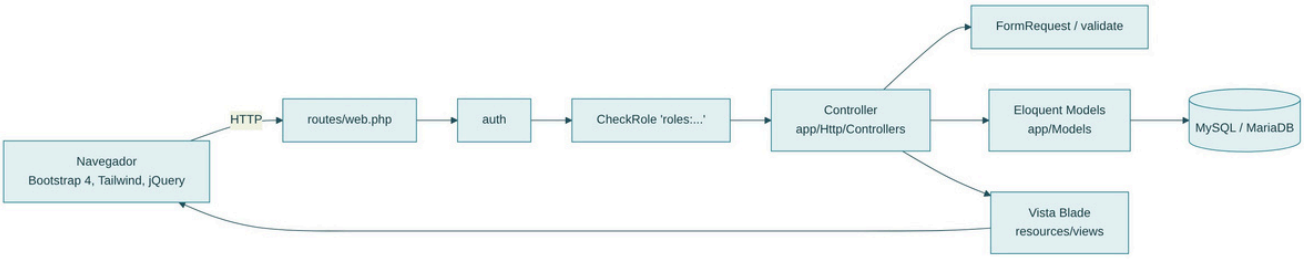


Figura 1. *Flujo de una petición HTTP en MedStats.*

3.2 Estructura de directorios

```
medstats/
├── app/
│   ├── Http/
│   │   ├── Controllers/          # 1 controlador por entidad (TrazabilidadController,
EstudioMedicoController...)
│   │   │   ├── Api/              # endpoints auxiliares (provincias, códigos postales,
procedimientos)
│   │   │   └── Auth/             # Breeze
│   │   ├── Middleware/CheckRole.php      # alias 'roles' (control de acceso por perfil)
│   │   └── Requests/StoreEstudioMedicoRequest.php
│   └── Models/                   # CajaQuirurgica, HistorialCaja, EstudioMedico, Estudio, Stock,
User...
├── database/
│   ├── migrations/               # esquema versionado (incluye migraciones de datos de perfilesy
modalidades)
│   └── seeders/                  # Pais, Provincia, CodigoPostal, Procedimiento
├── resources/
│   ├── views/trazabilidad/       # index, create, edit, show (línea de tiempo), estadisticas
│   ├── views/estudios_medicos/ # index, create, edit, show, estadisticas
│   ├── views/layouts/            # app.blade.php + _partials/sidebar, footer
│   └── js, css/                  # entradas de Vite
├── routes/web.php, auth.php
├── public/{assets,vendor,build}/
└── tests/{Feature,Unit}
```

3.3 Convenciones del proyecto

- Un **controlador por entidad** y un grupo de rutas por módulo, agrupadas por middleware.
- Nombres de rutas con la forma `modulo.accion` (`trazabilidad.estado`, `estudios_medicos.index`).
- Lógica sensible a la integridad (movimientos de stock, cambios de estado) dentro de `DB::transaction` o `DB::beginTransaction`.

- Mensajes de usuario en español; zona horaria `America/Argentina/Buenos_Aires` y ***locale*** `es` en `config/app.php` .

4. Seguridad y control de acceso

4.1 Modelo de perfiles

Cada usuario referencia un perfil mediante `users.role` (clave foránea lógica a `usuario_perfils.id`).
El perfil es una fila con **un indicador booleano por módulo**:

```
usuario_perfils: perfil, admin, insumos, estadisticas, pacientes, camas,
                 cirugias, estudios_medicos, trazabilidad, servicio_id
```

4.2 Middleware CheckRole

El alias `roles` (registrado en `app/Http/Kernel.php`) apunta a `CheckRole`. Recibe una lista de módulos y permite el paso si el perfil del usuario tiene **al menos uno** en verdadero; en caso contrario responde **403**.

```
Route::middleware(['auth', 'roles:admin,trazabilidad'])->group(function () {
    Route::get('/trazabilidad', [TrazabilidadController::class, 'index']);
    ...
});

// CheckRole::handle()
$perfil = UsuarioPerfil::find($user->role);
if (!$perfil) abort(403, 'Perfil no encontrado.');
foreach ($roles as $rol) {
    if (isset($perfil->$rol) && $perfil->$rol) return $next($request);
}
abort(403, "El perfil '{$perfil->perfil}' no tiene acceso a este módulo.");
```

4.3 Autorización dentro de controladores y vistas

`User::hasAccess($modulos)` acepta un string o un arreglo y devuelve ***true*** si el perfil habilita alguno. Se usa para (a) ocultar opciones en el sidebar y en las vistas y (b) reforzar el permiso dentro de acciones sensibles. Ejemplo: en `TrazabilidadController`, `create`, `store` y `destroy` exigen `admin`, mientras que `edit`, `update` y `actualizarEstado` aceptan `admin` o `trazabilidad`.

Además, `users.servicio_id` permite restringir a un usuario a un servicio (`User::servicioRestringido()`), lo que condiciona los listados de stock.

Perfil (migración)	admin	insumos	estadís - ticas	pacien - tes	camas	cirugías	estudios_ medicos	trazabi - lidad
Trazabilidad	0	1	1	0	0	1	0	1
Diagnóstico de Imagen	0	1	1	0	0	0	1	—
Administrador (a crear)	1	1	1	1	1	1	1	1

4.4 Otras medidas

- **CSRF**: todos los formularios incluyen `@csrf` y el método HTTP simulado con `@method`.
- **Validación en servidor** con `$request->validate()` o ***FormRequest***, con mensajes en español.
- **Contraseñas** con ***cast*** `hashed` y reglas `Rules\Password::defaults()`.
- **Bajas lógicas** (`SoftDeletes`) en cajas quirúrgicas para preservar la trazabilidad.

5. Base de datos

5.1 Diagrama entidad-relación · Trazabilidad

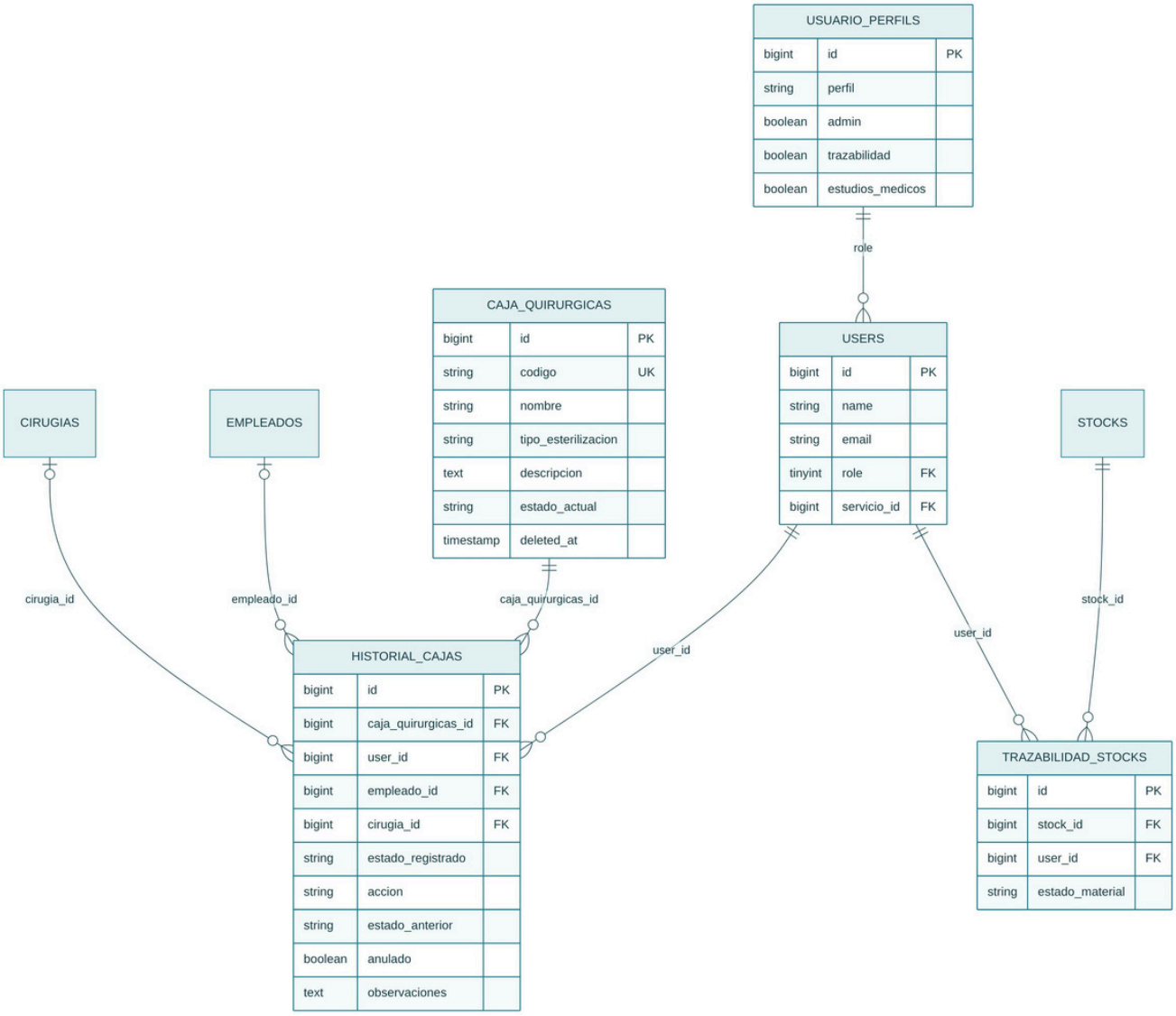


Figura 2. ***Entidades del módulo de trazabilidad y su relación con usuarios, empleados, cirugías y perfiles.***

5.2 Diccionario de datos · Trazabilidad

caja_quirurgicas

Columna	Tipo	Descripción
id	bigint PK	Identificador.
codigo	varchar(50) único	Código físico de la caja (ej. CAJ-001). La validación de formularios limita a 10 caracteres.
nombre	varchar(50)	Nombre descriptivo.

Columna	Tipo	Descripción
tipo_esterilizacion	varchar, null	***Autoclave*** u ***Óxido de Etileno***.
descripcion	text, null	Instrumental que contiene la caja.
estado_actual	varchar, def. 'Almacenada'	Estado vigente: Almacenada, En Uso, Esterilizada, Depósito Estér il.
deleted_at	timestamp, null	Baja lógica (estado «En Desuso»).
created_at / updated_at	timestamp	Auditoría temporal.

historial_cajas

Columna	Tipo	Descripción
caja_quirurgicas_id	FK → caja_quirurgicas (cascade)	Caja a la que pertenece el movimiento.
user_id	FK → users, null (nullOnDelete)	Usuario del sistema que realizó el movimiento.
empleado_id	FK → empleados, null	Empleado asociado (uso futuro; hoy se guarda null).
cirugia_ id	FK → cirugias, null	Cirugía donde se usó la caja (uso futuro).
estado_ registrado	varchar	Estado resultante del movimiento.
accion	varchar(20), def. 'avance'	**alta**, **avance**, **retroceso** (y **restauracion** contemplada en el scope).
estado_anter ior	varchar, null	Estado previo al movimiento.
anulado	boolean, def. 0	Marca los avances deshechos por un retroceso (no se bor ran).
obser vaciones	text, null	Texto descriptivo generado por el sistema.

trazabilidad_stocks

Tabla y modelo `TrazabilidadStock` (stock_id, user_id, ubicacion_origen, ubicacion_destino, estado_material, observaciones) previstos para trazar insumos individuales. Actualmente **no se utili- zan** desde controladores ni vistas.

5.3 Diagrama entidad-relación · Diagnóstico por Imágenes

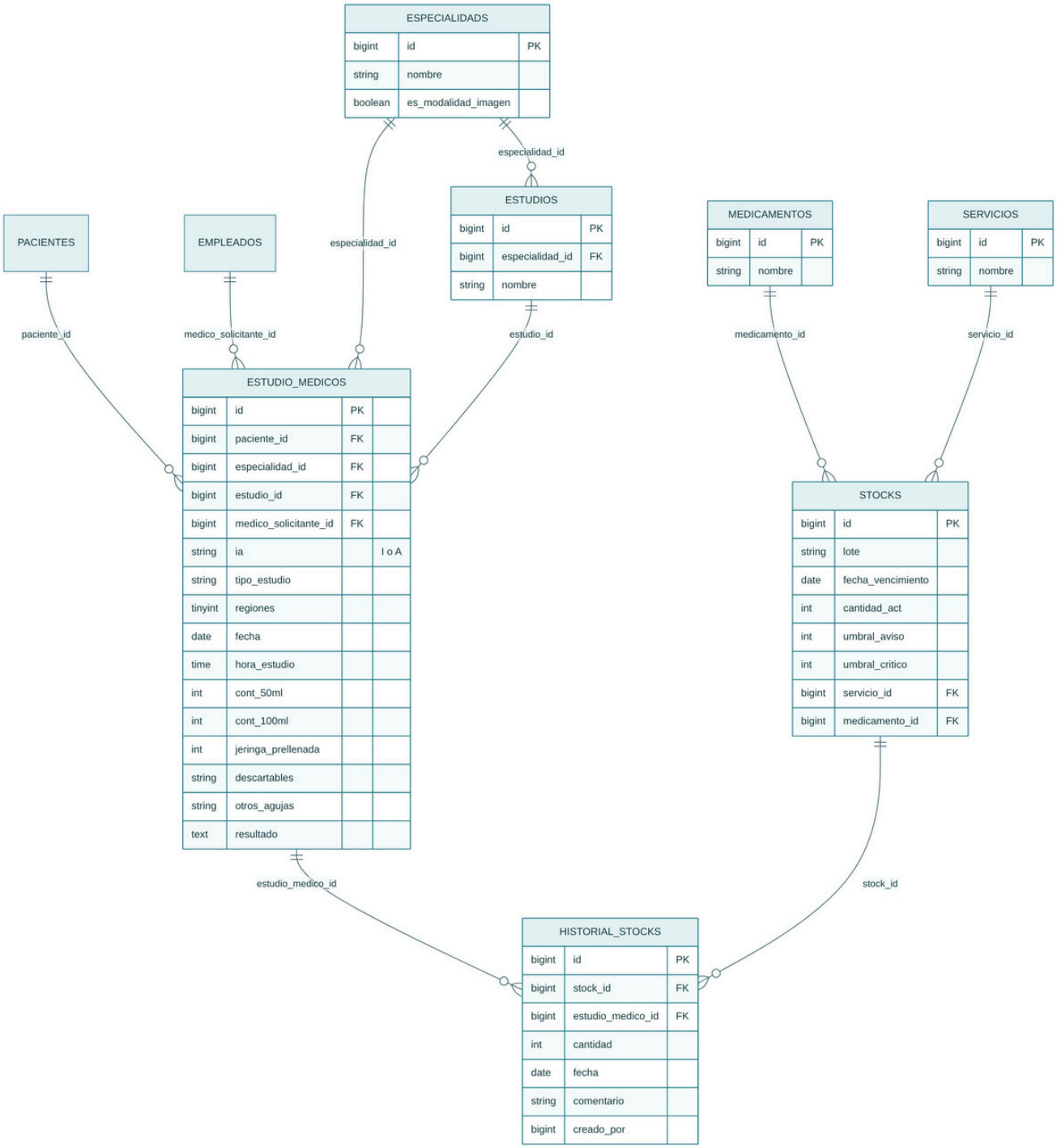


Figura 3. ***Entidades del módulo de diagnóstico por imágenes y su vínculo con el stock.***

5.4 Diccionario de datos · Diagnóstico por Imágenes

estudio_ medicos

Columna	Tipo	Descripción
paciente_id	FK → pacientes	Paciente estudiado.

Columna	Tipo	Descripción
medico_solicitante_ id	FK → empleados	Médico que solicita la práctica.
especialidad_id	FK → especialidads, null	Modalidad (solo con `es_modalidad_imagen = 1`).
estudio_ id	FK → estudios, null	Estudio real del catálogo.
tipo_estudio	varchar(150)	Copia del nombre del estudio al momento de guardar (se sincroniza en store/update).
ia	varchar(20), null	**I**nternado / **A**mbulatorio.
regiones	tinyint unsigned, null	Cantidad de regiones vistas (1–20). El valor anterior en texto libre se conserva en `regiones_legacy`.
fecha / hora_estudio	date / time, null	Momento del estudio.
cont_50ml, cont_100ml, jer inga_ prellenada	int, def. 0	Unidades de insumos fijos consumidas.
descartables, otros_agujas	varchar, null	Texto libre.
resultado	text, null	Observaciones / informe.

Tablas de soporte

Tabla	Rol en el módulo
especialidads	Se amplió con `es_modalidad_imagen`. La migración siembra «Rayos (X)» y «Tomografía». El scope `modalidadesImagen()` las filtra; `quirurgicas()` excluye.
estudios	Catálogo de estudios por modalidad. Índice único (especialidad_id, nombre).
medicamentos	Nombres de los insumos: «Contraste 50ml», «Contraste 100ml», «Jeringa Prellenada» (se crean automáticamente si no existen).
stocks	Lotes por servicio: cantidad_act, fecha_vencimiento, umbral_aviso (50) y umbral_critico (30) por defecto, barcode.
historial_stocks	Movimientos de stock; el campo `estudio_medico_id` vincula cada descuento o devolución con el estudio.
ser vicios	Debe existir «Diagnóstico por imágenes» (nombre exacto) para operar.
usuario_perfils	Se amplió con `estudios_medicos` y `trazabilidad`.

6. Módulo de Trazabilidad: implementación

6.1 Componentes

Componente	Archivo	Responsabilidad
Modelo	`app/Models/CajaQuirurgica.php`	***SoftDeletes***; relación `historiales()` con orden cronológico y solo movimientos vigentes.
Modelo	`app/Models/HistorialCaja.php`	Relaciones `cajaQuirurgica`, `empleado`, `user`; scope `enLinea`; accessor `responsable`; cast `anulado`.
Controlador	`TrazabilidadController`	CRUD de cajas, cambio de estado y estadísticas.
Vistas	`resources/views/trazabilidad/*`	Listado (DataTables + modales de contenido), formularios, línea de tiempo, estadísticas.
Indicadores	`InicioController`	KPI de cajas por método de esterilización y almacenadas en el inicio.

6.2 Métodos del controlador

Método	Ruta	Permiso	Descripción
index	GET /trazabilidad	admin | trazabilidad	Lista cajas; filtros `estado` y `nombre_caja[]`; «En Desuso» usa `onlyTrashed()`.
create / store	GET, POST /trazabilidad	admin (controlador)	Alta; valida y registra el historial con acción por defecto.
show	GET /trazabilidad/{id}	admin | trazabilidad	Línea de tiempo (`withTrashed`).
edit / update	GET, PUT /trazabilidad/{id}	admin | trazabilidad	Edición; si la caja estaba eliminada la restaura y la deja en «Almacenada».
destroy	DELETE /trazabilidad/{id}	admin (controlador)	Baja lógica.
actualizarEstado	POST /trazabilidad/{id}/estado	admin | trazabilidad	Parámetro `accion` = ***avanzar*** (default) o ***retroceder***.
estadisticas	GET /trazabilidad/estadisticas	admin | trazabilidad	Contadores y tabla de historial completo.

6.3 Máquina de estados

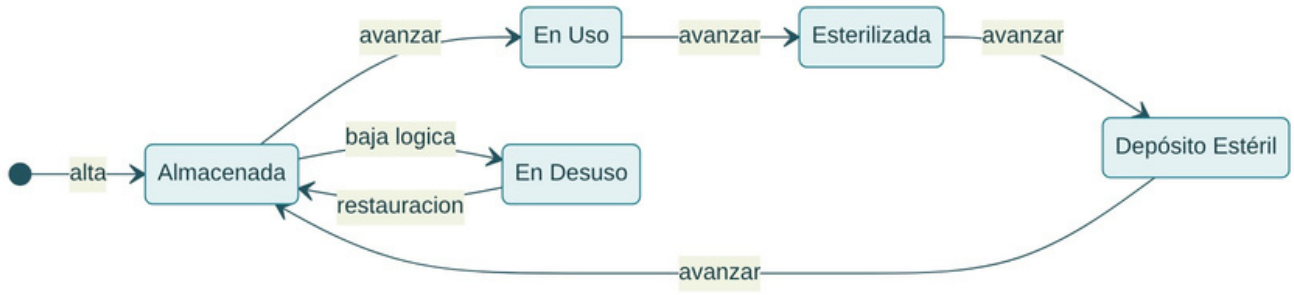


Figura 4. ***Transiciones válidas del ciclo de una caja.***

El avance recorre un ciclo cerrado definido en el controlador:

```
$flujo_normal = [
    'Almacenada'        => 'En Uso',
    'En Uso'            => 'Esterilizada',
    'Esterilizada'      => 'Depósito Estéril',
    'Depósito Estéril' => 'Almacenada',
];
$nuevo_estado = $flujo_normal[$estadoDesde] ?? 'Lavado';

DB::transaction(function () use (...) {
    $caja->update(['estado_actual' => $nuevo_estado]);
    HistorialCaja::create([
        'caja_quirurgicas_id' => $caja->id, 'user_id' => $usuario->id,
        'estado_registrado'     => $nuevo_estado, 'accion' => 'avance',
        'estado_anterior'       => $estadoDesde,
        'observaciones'         => 'Avance de "'.$estadoDesde.'" a "'.$nuevo_estado.'" por
'.$usuario->name,
    ]);
});
```

6.4 Auditoría: retroceso sin pérdida de información

El retroceso **no elimina** filas. Busca el último movimiento ***en línea*** (acción = avance), lo marca `anulado = true`, restaura el estado previo en la caja y agrega un registro con acción **retroceso**. La relación `historiales()` y el scope `enLinea` filtran por `accion IN ('alta','avance','restauracion') AND anulado = false`, por lo que la línea de tiempo muestra solo la historia vigente, mientras que `estadisticas()` expone el historial completo (incluyendo retrocesos y anulados).

```
public function scopeEnLinea($query)
{
    return $query->whereIn('accion', ['alta', 'avance', 'restauracion'])
                 ->where('anulado', false);
}
```

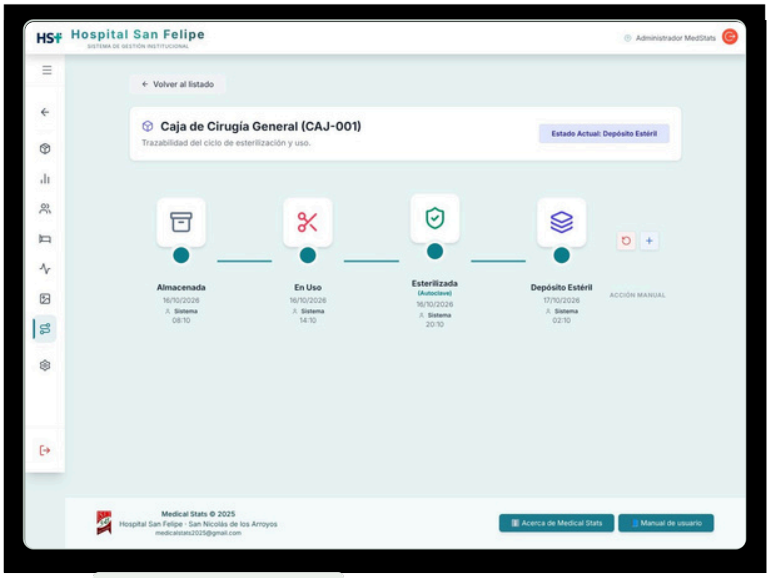


Figura 5. ***Vista*** `trazabilidad/show` ***: línea de tiempo y botones de acción manual.***

6.5 Vistas y comportamiento del cliente

- **Listado**: la tabla usa DataTables (orden, búsqueda y paginación en cliente). El filtro «Tipo de Caja» es un ***select*** múltiple enriquecido; el filtro «Estado» envía el formulario al cambiar.
- **Modal «Contenido»**: un modal Bootstrap por caja, renderizado en el servidor con el método de esterilización y la descripción (`white-space: pre-wrap`).
- **Línea de tiempo**: muestra `$caja->historiales->take(-4)`, es decir, los últimos cuatro movimientos vigentes. El retroceso se confirma con SweetAlert2 antes de enviar el formulario.
- **Estadísticas**: siete contadores y tabla de historial con `with(['cajaQuirurgica','empleado','user'])` .

6.6 Cómo extenderlo

- **Nuevo estado en el ciclo:** agregarlo al arreglo `$flujo_normal` del controlador y a los mapas `$flujo_normal` / `$flujo_inverso` y los íconos de `show.blade.php`; sumarlo al filtro de `index.blade.php` y a las tarjetas de `estadisticas`. Se recomienda centralizar la lista de estados en una constante del modelo.
- **Nuevo método de esterilización:** ampliar la regla `in:Autoclave,Óxido de Etileno` en `store` y `update`, los ***select*** de `create/edit` y los contadores de `InicioController`.
- **Vincular una caja a una cirugía:** las columnas `cirugia_id` y `empleado_id` de `historial_cajas` ya existen; basta con completarlas al registrar el movimiento.

7. Módulo de Diagnóstico por Imágenes: implementación

7.1 Componentes

Componente	Archivo	Responsabilidad
Modelos	`EstudioMedico`, `Estudio`, `Especialidad`	Relaciones paciente, médico, modalidad, estudio y `movimientos_stock()` .
Controlador	`EstudioMedicoController`	CRUD, endpoints auxiliares, estadísticas y lógica de stock.
Validación	`StoreEstudioMedicoRequest`	Reglas del alta; autoriza con `hasAccess('estudios_medicos')` .
Catálogo	`EstudioController` (resource)	ABM de estudios en Ajustes (rol admin).
Vistas	`resources/views/estudios_medicos/*`	Listado, formulario, ficha, edición y estadísticas.

7.2 Rutas y endpoints auxiliares

```
Route::middleware(['auth', 'roles:estudios_medicos'])->group(function () {
    Route::get('/estudios-medicos/estadisticas', ...)->name('estudios_medicos.estadisticas');
    Route::resource('estudios-medicos', EstudioMedicoController::class)
        ->names('estudios_medicos')->parameters(['estudios-medicos' => 'estudio_medico']);
    Route::get('/estudios-medicos/especialidad/{especialidad}/estudios',
'estudiosPorEspecialidad');
    Route::get('/estudios-medicos/paciente/{paciente}/internacion', 'internacionPaciente');
});
```

- `estudiosPorEspecialidad` devuelve JSON `[{id, nombre}]` para el ***select*** encadenado Modalidad → Estudio (Select2 + ***fetch*** desde la vista).
- `internacionPaciente` devuelve `{internado, cama, habitacion, sala, desde}` a partir de `ocupacion_camas` con `fecha_egreso IS NULL`; alimenta el panel de cama cuando se marca «Internado».

7.3 Validación

```
'especialidad_id' => ['required','integer',
        Rule::exists('especialidads','id')->where(fn($q) => $q->where('es_modalidad_imagen',
true))],
'estudio_id'              => ['required','integer',
        Rule::exists('estudios','id')->where(fn($q) => $q->where('especialidad_id', $this-
>input('especialidad_id')))],
'regiones'                => ['nullable','integer','min:1','max:20'],
'fecha'                   => ['required','date','before_or_equal:today'],
'hora_estudio'            => ['nullable','date_format:H:i','before_or_equal:now'],
'cont_50ml'               => ['required','integer','min:0'],              // idem cont_100ml, jeringa_prellenada
```

La regla anidada garantiza que el estudio pertenezca a la modalidad elegida, aun si se manipula el formulario.

7.4 Descuento transaccional de insumos

Los tres insumos se identifican por **nombre** mediante constantes del controlador y se descuentan del servicio `«Diagnóstico por imágenes»`:

```
private const SERVICIO_DIAGNOSTICO_IMAGEN = 'Diagnóstico por imágenes';
private const INSUMO_CONTRASTE_50 = 'Contraste 50ml';
private const INSUMO_CONTRASTE_100 = 'Contraste 100ml';
private const INSUMO_JERINGA              = 'Jeringa Prellenada';
```

El flujo de `store()` es el siguiente:

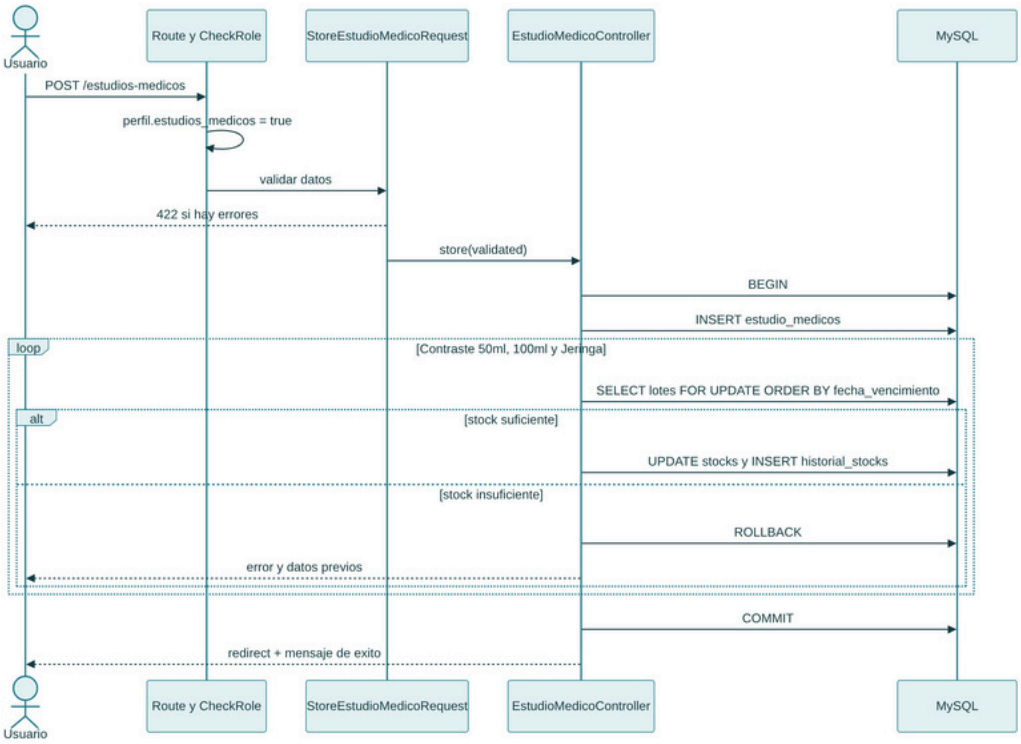


Figura 6. ***Secuencia del alta de un estudio con descuento de stock.***

1. **Transacción:** `DB::beginTransaction()` envuelve el ***insert*** del estudio y los tres descuentos.

2. **Selección de lotes (FEFO)** `Stock::where(medicamento, servicio)->where('cantidad_act','>',0)->orderBy('fecha_vencimiento')->lockForUpdate();` se consumen primero los lotes que vencen antes.
3. **Control:** si el total disponible es menor al pedido se lanza una excepción con el detalle; el ***catch*** hace `rollBack()` y vuelve al formulario con `withErrors` y `withInput`.
4. **Registro:** por cada lote afectado se decrementa `cantidad_act` y se crea un `Historial_stock` con cantidad negativa, `estudio_medico_id`, paciente, médico y `creado_por = auth()->id()`.

Edición: `procesarDeltaInsumo()` calcula `nueva − original`; si es positiva descuenta, si es negativa llama a `devolverInsumoFijo()`. **Eliminación:** devuelve las tres cantidades antes de borrar el estudio. `devolverInsumoFijo()` reintegra primero a los lotes originalmente afectados (consultando `historial_stocks`) y, si el lote ya no existe, al lote vigente con mayor vencimiento.

> **IMPORTANTE**
>
> Los nombres de insumos y del servicio actúan como **claves de negocio**. Si se renombran en la base de datos, el descuento fallará. Una mejora recomendada es referenciarlos por ID o por una tabla de configuración.

7.5 Estadísticas

`estadisticas()` valida filtros (`desde`, `hasta`, `anio`, `especialidad_id`, `medico_id`) y construye una consulta base reutilizable que se clona para cada indicador: total y promedios (mensual = total / (días/30,4375)), estudios por mes (`DATE_FORMAT(fecha,'%Y-%m')`), top de médicos, top de estudios (con suma de regiones), reparto por modalidad, carga por día (`WEEKDAY`), internados/ambulatorios y consumo de contraste. Los resultados se pasan a la vista, que los dibuja con Chart.js.

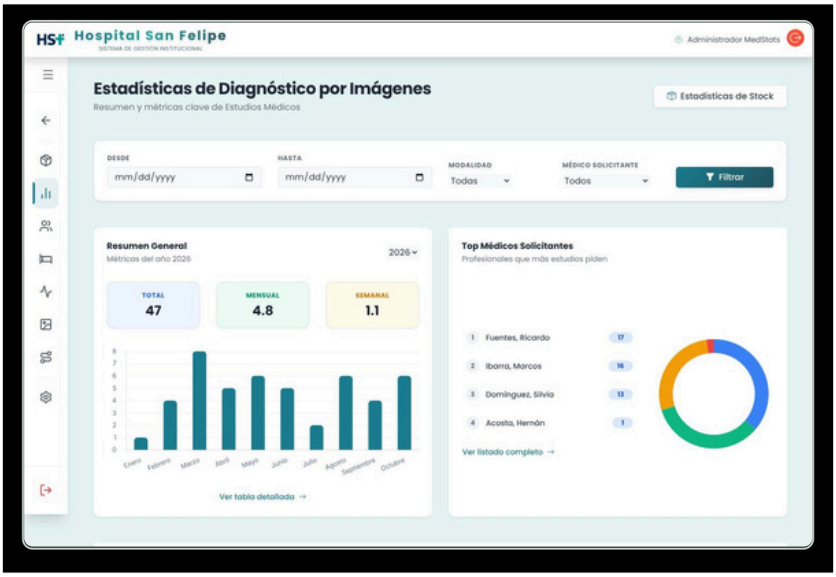


Figura 7. ***Vista de estadísticas con filtros y gráficos.***

7.6 Exportación

El listado incluye DataTables con botones ***Imprimir***, ***PDF*** (jsPDF + AutoTable) y ***Excel*** (SheetJS), con la opción de exportar «solo esta página» o «todo lo filtrado». La exportación se hace en el cliente; no hay endpoint de reportes en el servidor.

7.7 Cómo extenderlo

- **Nueva modalidad** (por ejemplo, Ecografía): crear la especialidad con `es_modalidad_imagen = 1` desde Ajustes → Especialidades/Estudios; no requiere cambios de código.
- **Nuevo insumo controlado:** agregar una constante, una columna en `estudio_medicos`, el contador en ***create/edit***, las reglas del ***FormRequest*** y las llamadas a `descontarInsumoFijo` / `procesarDeltaInsumo` / `devolverInsumoFijo`.

8. Mapa de rutas

8.1 Trazabilidad (middleware auth + roles:admin,trazabilidad)

Método	URI	Nombre	Acción
GET	/trazabilidad	trazabilidad.index	Listado
GET	/trazabilidad/estadisticas	trazabilidad.estadisticas	Estadísticas e historial
GET	/trazabilidad/create	trazabilidad.create	Formulario de alta
POST	/trazabilidad	trazabilidad.store	Guardar caja
GET	/trazabilidad/{id}	trazabilidad.show	Línea de tiempo
GET	/trazabilidad/{id}/edit	trazabilidad.edit	Formulario de edición
PUT	/trazabilidad/{id}	trazabilidad.update	Actualizar / restaurar
DELETE	/trazabilidad/{id}	trazabilidad.destroy	Enviar a desuso
POST	/trazabilidad/{id}/estado	trazabilidad.estado	Avanzar / retroceder

8.2 Diagnóstico por Imágenes (auth + roles:estudios_medicos)

Método	URI	Nombre	Acción
GET	/estudios-medicos	estudios_ medicos.index	Listado
GET	/estudios-medicos/create	estudios_ medicos.create	Formulario
POST	/estudios-medicos	estudios_medicos.store	Guardar y descontar stock
GET	/estudios-medicos/{estudio_ medico}	estudios_ medicos.show	Ficha
GET	/estudios-medicos/{estudio_ medico}/edit	estudios_ medicos.edit	Edición
PUT/PATCH	/estudios-medicos/{estudio_ medico}	estudios_ medicos.update	Actualizar y ajustar stock
DELETE	/estudios-medicos/{estudio_ medico}	estudios_ medicos.destroy	Eliminar y devolver stock
GET	/estudios-medicos/estadisticas	estudios_medicos.estadisticas	Tablero
GET	/estudios-medicos/especialidad/{especialidad}/estudios	estudios_ medicos.estudios_ por _especialidad	JSON de estudios

Método	URI	Nombre	Acción
GET	/estudios-medicos/paciente/{paciente}/internacion	estudios_medicos.internacion_paciente	JSON de internación

8.3 Administración y soporte (roles:admin salvo indicación)

Recurso	Rutas	Observación
Estudios (catálogo)	`Route::resource('estudios')` sin ***show***	ABM del catálogo por modalidad.
Especialidades	/especialidades (CRUD)	Incluye modalidades de imagen.
Ser vicios	`Route::resource('servicios')`	Debe existir «Diagnóstico por imágenes».
Usuarios / Perfiles	/usuarios, /perfiles, PUT /usuar ios/{id}/actualizar-rol	Asignación de perfiles.
Insumos (stock)	/stocks (roles:insumos,estudios_medicos)	Lotes, escaneo de código de barras.
Medicamentos	/medicamentos (roles:admin,insumos)	Nombres de insumos.

9. Capa de presentación

- **Layout principal** `layouts/app.blade.php`: carga los estilos y scripts locales, incluye el ***sidebar***, la cabecera con el usuario y el pie con «Acerca de» y **Manual de usuario** (`public/assets/files/Manual_Usuario_MedStats.pdf`).
- **Sidebar** `layouts/_partials/sidebar.blade.php`: arreglo `$menuItems` con ruta, ícono, permiso (`access`) y rutas que lo resaltan (`match`/`except`). Un nuevo módulo se agrega como un elemento más del arreglo.
- **Estilo**: paleta institucional (`#1B7D8F`, `#2BA8A0`, `#245360`), tipografías locales (Figtree, Poppins, Inter) y tablas con cabecera verde azulada.
- **Íconos** Lucide (`data-lucide="..."`) inicializados con `lucide.createIcons()`.
- **Errores**: vistas personalizadas en `resources/views/errors` (403, 404, 419, 429, 500, 503).

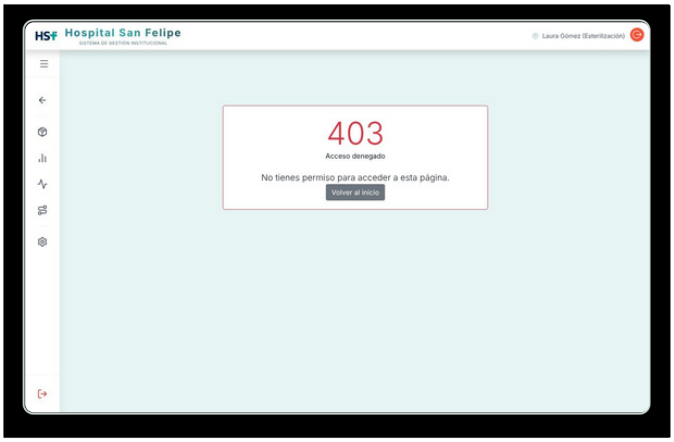


Figura 8. ***Vista de error 403 personalizada.***

10. Instalación y despliegue local

10.1 Requisitos

- PHP 8.1 o superior con extensiones `mbstring,openssl,pdo_mysql,tokenizer,xml, ctype, fileinfo, json.`
- Composer 2, Node.js 18+ (el repositorio incluye Node 22 para Windows) y npm.
- MySQL 8 o MariaDB 10.5+ (Laragon los incluye) y Git.

10.2 Pasos

1. Clonar la rama de trabajo:

```
git clone -b trazabilidad https://github.com/AgustinPonceDev/MedStats.git
cd MedStats
```

2. Instalar dependencias PHP y JS:

```
composer install
npm install
```

3. Crear el archivo `.env` en la raíz (el repositorio no incluye `.env.example`):

```
APP_NAME=MedStats
APP_ENV=local
APP_DEBUG=true
APP_URL=http://127.0.0.1:8000
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=medstats
DB_USERNAME=root
DB_PASSWORD=
SESSION_DRIVER=file
CACHE_DRIVER=file
```

4. Crear la base vacía (`CREATEDATABASEmedstats CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;`) y generar la clave:

```
php artisan key:generate
```

5. Migrar y cargar los datos base (países, provincias, códigos postales, procedimientos):

```
php artisan migrate
php artisan db:seed
```

6. Compilar los ***assets*** y levantar el servidor:

```
npm run build          #o:npmrun dev
php artisanserve       #oelvirtual host de Laragon
```

10.3 Datos mínimos para operar los nuevos módulos

Las migraciones crean los perfiles ***Trazabilidad*** y ***Diagnóstico de Imagen*** y marcan «Rayos (X)» y «Tomografía» como modalidades. Falta crear un administrador, el servicio y los insumos. Desde `php artisan tinker`:

```
$p = App\Models\UsuarioPerfil::create(['perfil'=>'Administrador','admin'=>1,'insumos'=>1,

'estadisticas'=>1,'pacientes'=>1,'camas'=>1,'cirugias'=>1,'estudios_medicos'=>1,'trazabilidad'=>1]);
App\Models\User::create(['name'=>'Administrador','email'=>'admin@hospital.local',
        'password'=>'CambiarEstaClave123','role'=>$p->id]);

$s = DB::table('servicios')->insertGetId(['nombre'=>'Diagnóstico por imágenes',
        'created_at'=>now(),'updated_at'=>now()]);
foreach (['Contraste 50ml','Contraste 100ml','Jeringa Prellenada'] as $n) {
      $m = DB::table('medicamentos')-
>insertGetId(['nombre'=>$n,'created_at'=>now(),'updated_at'=>now()]);
      DB::table('stocks')->insert(['lote'=>'L-
'.strtoupper(substr($n,0,3)).'-001','cantidad_act'=>100,
          'fecha_vencimiento'=>now()->addYear(),'servicio_id'=>$s,'medicamento_id'=>$m,
          'created_at'=>now(),'updated_at'=>now()]);
}
```

Luego ingrese con ese usuario, cree los estudios del catálogo desde **Ajustes → Estudios** y los usuarios operativos desde **Ajustes → Usuarios** asignándoles perfil en **Ajustes → Perfiles**.

> **IMPORTANTE** Los usuarios creados por ***Registro*** (`/register`) no tienen perfil (`role = null`) y reciben 403 hasta que un administrador se los asigne. Cambie la contraseña del administrador inicial antes de poner el sistema en producción.

10.4 Publicar el manual de usuario

El pie de página enlaza a `public/assets/files/Manual_Usuario_MedStats.pdf`. Para publicar la versión actualizada basta con reemplazar ese archivo y confirmarlo en Git.

10.5 Despliegue en producción (lineamientos)

- `APP_ENV=production`, `APP_DEBUG=false`, HTTPS obligatorio y ***document root*** apuntando a `public/`.
- `composer install --no-dev --optimize-autoloader`, `npm run build`, `php artisan config:cache` `route:cache view:cache`.
- Permisos de escritura solo en `storage/` y `bootstrap/cache/`; copias de seguridad programadas de la base (el historial de cajas es evidencia de auditoría).

11. Estándares de codificación y flujo de trabajo

Tema	Criterio
Estilo PHP	PSR-12, aplicado con `vendor/bin/pint` antes de cada ***commit***.
Formato de archivos	`.editorconfig`: UTF-8, fin de línea LF, indentación de 4 espacios, salto de línea final.
Nombres	Clases en ***PascalCase***, métodos y variables en ***camelCase***, tablas en plural y ***snake_case***, vistas en carpeta por módulo.
Controladores	Delgados: validar, delegar y redirigir. Lógica de negocio compleja en métodos privados o ser vicios.
Integridad	Operaciones multi-tabla dentro de transacción; bloqueo pesimista (`lockForUpdate)` al tocar stock.
Migraciones	Una por cambio, con `up` y `down` reversibles; nunca editar migraciones ya aplicadas.
Seguridad	Todo formulario con `@csrf`; toda ruta dentro de `auth` + `roles`; validar siempre en servidor.
Ramas	`trazabilidad` es la rama de integración actual; trabajar en ramas de funcionalidad y fusionar mediante ***pull request***.
Commits	Mensajes en español, en presente y descriptivos (qué y por qué).
Mensajes de usuar io	Siempre en español, claros y accionables.

12. Pruebas

El proyecto incluye PHPUnit 10 con la configuración `phpunit.xml` y las pruebas de ejemplo de Breeze (`tests/Feature/Auth`, `ProfileTest`). Se ejecutan con `php artisan test`.

Casos prioritarios para incorporar en los nuevos módulos:

- **Trazabilidad:** ciclo completo de cuatro avances; retroceso que anula el último avance; imposibilidad de retroceder el estado inicial; permisos por perfil (403); restauración de una caja en desuso.
- **Diagnóstico por Imágenes:** alta con descuento FEFO entre dos lotes; ***rollback*** ante stock insuficiente; edición que aumenta y reduce insumos; eliminación que devuelve stock; rechazo de un estudio que no pertenece a la modalidad.

```
public function test_no_permite_retroceder_el_estado_inicial(): void
{
      $this->actingAs($this->adminConPerfil());
      $caja = CajaQuirurgica::create([...'estado_actual' => 'Almacenada']);
      $this->post(route('trazabilidad.estado', $caja), ['accion' => 'retroceder'])
            ->assertSessionHas('error');
}
```

> **NOTA**
>
> Como varias consultas estadísticas usan funciones de MySQL, las pruebas deben ejecutarse contra MySQL/MariaDB (no SQLite en memoria).